Open API Testing Using Large Language Models and Prompt Engineering

Phyu Thet Thet Kyaw
Department of Mathematics and Computer Science
Chulalongkorn University
Bangkok, Thailand
phyusnoe846@gmail.com

Dr. Arthorn Luangsodsai
Department of Mathematics and Computer Science
Chulalongkorn University
Bangkok, Thailand
arthorn.l@chula.ac.th

Dr. Pattarasine Bhattarakosol
Department of Mathematics and Computer Science
Chulalongkorn University
Bangkok, Thailand
pattarasinee.b@chula.ac.th

2025 22nd International Joint Conference on Computer Science and Software Engineering (JCSSE) | 979-8-3315-7358-4/25/$31.00 ©2025 IEEE | DOI: 10.1109/JCSSE67377.2025.11297930

Abstract— Large Language Models have emerged as the top-notch tool in the software engineering field, from requirement gathering and analysis to code generation. Several approaches have been developed to apply artificial intelligence in software testing, from using the fine-tuned language models in documentation parsing to test case generation. However, these approaches require special tools and interfaces, which can take time to adopt in real-world software testing. In this paper, we aim to explore the capacity and performance of prevalent Large Language Models in generating API test cases and import them to existing testing tools because API testing is one of the core and earliest phases to ensure the quality and matching requirements of developed software. Natural language processing and code generation capabilities of four state-of-the-art LLMs, Grok, Claude, Gemini, and GPT, are applied to generate API test cases, covering the path, parameter, and response checking coverage, which are the main features of API testing. It was discovered that native LLMs can be applied in API testing effectively, given the API document and prompted with clear and concise information. All models under study can properly understand the natural language description in the API document and call the endpoint with appropriate parameter values according to the description. Among all four models, Grok and Claude showed the best performance with Chain-of-Thought Prompting approach.

Keywords— API, Test Automation, Large Language Models

I. INTRODUCTION

Nowadays, software is mainly developed as loosely coupled services, adopting Microservices Architecture (MSA), starting from the leading tech companies like Netflix and Amazon [1]. The main core of MSA software is the Application Programming Interface (API), which serves as a contract between applications or services, exposing only abstract services and hiding complex internal implementations for easy integration. Representation State Transfer (REST) API has become the standard approach of API development in the current industry and is documented using Open API or Swagger format. Comprehensive testing of these services, especially in the current trend of rapid software delivery approaches, imposes a significant challenge, requiring extensive effort and knowledge of the API architecture and its intended functionality.

Thus, automated testing has become an interesting topic in the realm of software engineering, and numerous works on test automation have been conducted, ranging from automated test data generation, Graphical User Interface (GUI) testing, to API testing. In the field of API testing, automated testing tools like RESTler [2] have evolved, which analyzes a Swagger document and automatically generates test sequences using search strategies, like Breadth-First-Search (BFS), BFS-Fast, and Random walk. Another approach [3] applied model-based specification-based techniques for API testing using OpenAPI documentation by representing both the API document and test cases as models and then transforming these models into executable test code using JUnit. NLPtoREST [4] extracts the human-readable parts of the OpenAPI specification and uses pre-trained Word2Vec models to generate enhanced documentation to improve test coverage.

Artificial Intelligence has also played a great role in the software engineering field, particularly the Large Language Models (LLM), which have been regarded as cutting-edge Artificial Intelligence (AI) systems. Due to their Natural Language Processing, Contextual Awareness, and code generation capabilities, their applications extend from daily use as a personal assistant in email drafting to medical applications as in medical decision support systems [5]. Fine-tuned LLMs are also widely used in software engineering, like code auto-completion by GitHub Copilot and AI-assisted coding with context awareness in Cursor. In the case of software testing, the use of AI has been proven efficient by studies like Agent-based Software Testing with LLM [6], which captures user input as voice, processes the prompt, and executes tasks for testing the source code using the underlying LLMs like GPT and CodeLlama and the Selenium testing framework. Besides, the prompt engineering and LLMs have also proven potential for API testing, as in Proptest-AI [7] by extracting information from API documents and generating test cases by the language models.

In this paper, the following contributions are made:

(1) Four popular LLMs are prompted with 4 different approaches to generate test cases which cover the path, parameter and response validation of public APIs.

(2) Performance evaluation of those models against each prompting approach.

The remainder of this paper is structured as follows. Section 2 delves into the theoretical background of API testing and the significance of prompt engineering in enhancing the performance of LLMs. Section 3 presents a comprehensive review of previous research on automated testing and the integration of AI and LLMs in API testing. Section 4 outlines the proposed approach of this paper, including the data collection and experiment setup processes. Section 5 provides a concise summary of the key findings obtained from the study, while Section 6 concludes the study by summarizing its most significant conclusions.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on April 23,2026 at 16:58:20 UTC from IEEE Xplore. Restrictions apply.

II. Background

A. Application Programming Interface (API)

Application or software is now the center of daily lives, and thus there is a need for applications that offer a wide variety of features to consumers. As the use of applications has become widespread, the software is also becoming more and more complex. In modern software development, API is the main core for two main reasons. Firstly, it serves as the connector between different services and applications, enhancing the scalability and modularity of applications. Secondly, API ensures rapid software delivery by hiding the complex implementation of each service, which allows the developers to integrate and design software more efficiently. API can be mainly grouped into 2 categories: web APIs for internet communication and library APIs for software libraries. In the case of Web APIs, there are several subtypes like SOAP, which rely on XML messaging, and REST API, which operates on standard HTTP methods [8]. According to recent studies [9], 70% of GitHub repositories use REST API as the standard architecture.

REST was first introduced by Roy Fielding in 2000 [10]. REST API maps HTTP methods with resources as CRUD operations: POST to create a resource, GET to retrieve the resource, PUT to update the resource, DELETE to remove the resource.

B. Open API Documents

These REST APIs are documented as Open API specification (formerly known as Swagger definition) in JSON or YAML format, whose latest version is 3.x.x. According to the official website [11], "the OpenAPI Specification (OAS) defines a standard, language-agnostic interface to HTTP APIs which allows both humans and computers to discover and understand the capabilities of the service without access to source code". As in Figure 1, the available resources and operations are documented under the paths attribute, which is the most important part of the document. It contains the available URI (e.g., '/geomarks'), the operation methods (e.g., "post"), the description of parameters, and the response status code for indicating the status of the endpoint call (e.g., "200"/ "302").

C. API Testing

Web service APIs can be tested using either a black-box approach [4] [12] or a white-box approach [13]. Black-box testing is usually based on the API documents, ensuring all elements and defined functions are covered by the test suite, while white-box testing usually focuses on source-code coverage, like line coverage and branch coverage [14]. The evaluation metrics for white-box testing are mainly the source-code coverage, while the coverage criteria for black-box testing were proposed by [14] in terms of input coverage and output coverage.

D. Large Language Models (LLMs)

LLMs are a type of language models which include neural networks with billions of parameters, trained on vast amount of unlabeled raw text data in a self-supervised learning approach [15]. LLMs have proved their ability in various language processing tasks, including text generation, translation, and sentiment analysis, by leveraging deep learning techniques and large datasets [16]. LLMs have been applied in wide varieties of tasks from data analysis and information extraction in different fields like finance, market research, content generation for social media posts, AI-assisted coding to decision making in medical diagnosis [17].

E. Prompt Engineering

According to the paper by[18], "A prompt is a piece of text or instruction given to the model to get a desired response. Prompt engineering is the practice of creating effective inputs (prompts) to guide the behavior and output of Large Language Models". Prompt engineering is becoming a crucial skill in maximizing the performance of LLMs across various applications [19]. According to a study [20], a prompt consists of four main components to define the task for LLM and provide the necessary information to generate the desired outcome.

- Instruction- tells the model what to do (e.g., "Proofread the text");
- Context and Input indicator - offers background to help the model respond appropriately (e.g., "This is a formal corporate email");
- Output indicator - clarifies the expected format or focus of the response (e.g., "Only fix grammar and punctuation, don't rewrite").

There are different types of prompts based on the examples or reasoning strategies they use [21]. Zero-shot prompts provide no examples and rely on the model's general knowledge (e.g., "Translate this sentence to French"). One-shot prompts include a single example to guide the model. Few-shot prompts give several examples to establish a clearer pattern for the model to follow. Chain-of-thought prompts encourage step-by-step reasoning by prompting the model to explain its thought process before reaching a conclusion. Each of these approaches can be used strategically to improve the accuracy, reliability, and interpretability of AI-generated responses.

III. Related Work

Several works have been performed on automatic testing of REST API. A popular tool or research on white-box testing is the EvoMaster by Acuri [13], which is a system-level testing of web APIs which utilizes optimization algorithms like

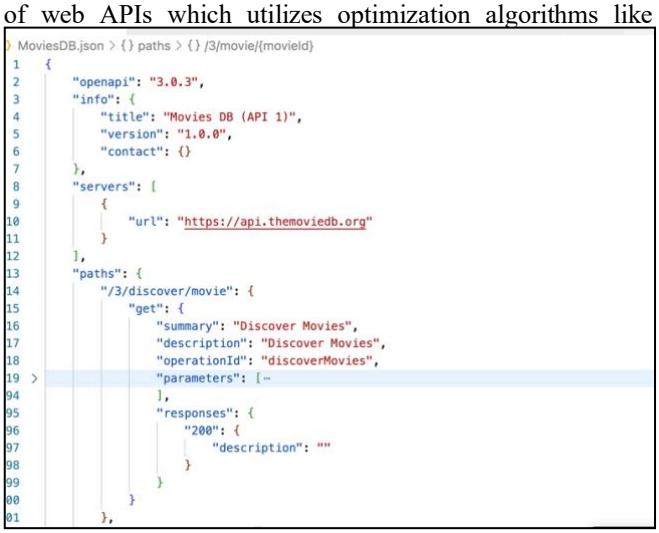


Fig. 1. Example snippet of an Open API documentation

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on April 23,2026 at 16:58:20 UTC from IEEE Xplore. Restrictions apply.

Evolutionary Algorithm to generates test cases (HTTP endpoint calls) based on the OpenAPI specification

RestTestGen, in Java language, [22] proposes the use of an Operation Dependency Graph (ODG) which encodes data dependencies among the operations of an API. For example, the request GET /products/{productId} can retrieve the product ID from the response of previous GET /products. The ODG can be inferred from the OpenAPI specification's parameter and schemas and improves test coverage by prioritizing the endpoint request orders. Reference [23] also proposes functional testing of REST APIs, with four configurations for test case generation, from fully automated to semi-automated approach. This technique is designed with Java and requests manual user input for parameters that have no detailed values in the OpenAPI specification. It parses the API documentation (in JSPN format) into models, which are then executed using Eclipse IDE. bBOXRT [22] is a black-box robustness testing tool for REST APIs, implemented in Java and based on the OpenAPI documents. The tool has 2 modules: Workload generator analyzes the document and generates valid requests to understand the service under test while the Fault load generator applies mutations and generates invalid requests to find the faulty interactions, which are further analyzed manually by test engineers. RESTest [24] also generates both nominal and faulty test cases focusing on inter-parameter dependencies support. For example, in the YouTube search API parameters, the publishedAfter parameter must be greater or equal to the publishedBefore parameter. RESTest generates test cases using Random Testing (nominal test cases) from the specification and Constraint-based Testing by exploiting constraints of inter-parameter dependencies from the nominal test cases.

Property-based testing by LLM [25] has also recently been conducted on Python libraries using proprietary models like OpenAI's GPT-4, Anthropic's Claude-3-Opus, and Google's Gemini-1.5-Pro. Another approach by [26] performed the GUI interface testing and evaluated the performance of test reports by senior manual testers and the report from LLMs on source codes.

IV. Proposed Approach

This study aims to evaluate the natural language processing and code generation capability of several LLMs in software testing field. API testing either requires the actual source code or documentation as the input for test case generation. We aim to employ **black-box testing approach** because in real-world scenarios, the actual source code is usually limited only to developers. The general concept of the approach is summarized in Figure 2.

A. Data Preparation and Collection

The Open-source APIs will be collected from websites like like API gurus [27], GitHub API [28] , Rapid API [29] and Public APIs [30]. Ideally, an API should contain at least 2 resources, each of which can have three kind of operations (create, retrieve, update operations) since some open-source API will not allow delete operations on the resources. API

documents of 7-10 [25] [26] systems will be collected as sufficient sample size to prove each LLM's consistent performance across different technical domains, parameter

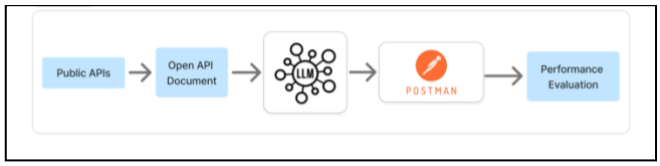


Fig. 2. Overview of proposed approach

values and description, providing a robust benchmark for evaluating their comprehension and reasoning capabilities.

Afterwards, these APIs will be manually called in Postman tool and generate an Open API specification using the official Postman to Open API package [31]. The generated document will be manually analyzed to ensure that they are confined to basic Open API specifications, which means the document should contain basic structure such as metadata, info, servers, and properties in a correct format.

B. Exploration and selection of Large Language Models (LLMs)

Public (free) versions of existing LLMs, such as **OpenAI's GPT-4-turbo, Anthropic's Claude Sonnet 4, Google's Gemini 2.5 Flash, xAI's Grok** will be explored to generate test cases. The **API specification in json or yaml format** will be fed to the model, and the model will be prompted to generate executable test cases.

C. Test case generation

The test cases are HTTP calls to the API service, targeting each endpoint with required configuration and parameters. One successful HTTP call to an endpoint will be regarded as one test case meaning that test case should be generated in correct Postman compatible format. The output of these models depends heavily on the prompting techniques used. Four different prompt structures will be defined to feed into each model, such as **Zero shot, One Shot, Few Shot and Chain-of-thought Prompting**.

In order to create successful API calls, the LLM needs to extract information such as

- the base URL and configuration of the service,

- the API endpoints defined under 'paths' properties, including the available operations (Create, Retrieve, Update, Delete) on the endpoint, the required parameters and path variables defined for each endpoint.

- the sequence of API calls identifiable from URI and operations type

In Zero shot prompting, the model will be simply fed with the API specification and tasked to generate test cases by learning the documentation itself.

For **One Shot and Few Shots prompting**, the model will be explained about the structure of the API, giving examples of how to set the dynamic values in the API by indicating the properties and description in the documentation.

In **Chain-of-though prompting**, model will be further prompted to get complete coverage of the API, indicating the sequence in which the endpoints should be called. For example, the model will be prompted to call 'Get all' endpoints and after that, the endpoint for a specific resource can be called by getting the values from previous endpoint's

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on April 23,2026 at 16:58:20 UTC from IEEE Xplore. Restrictions apply.

responses. These values can be dynamically set as collection variables in Postman.

D. Experiment set up and Test execution

The test cases are generated in JSON format by LLM and can be directly imported to Postman testing tool. Each import will be created as a Postman Test Collection. Each test collection will contain a set of API requests. An API request is regarded as a test case against each endpoint. A request will contain one of CRUD operations on an endpoint, the base path of the service, the relative path, query parameters, path variables and or body parameter of each endpoint depending on the specification. In terms of Authentication of these services, they would be manually set by the engineer because authentication and authorization tokens should not be exposed to publics.

The performance of LLMs is evaluated on whether the model can generate the test case with appropriate values for those components as defined in the document, rather than with some random values. The generated test collections will then be run manually with Postman Collection Runner. After each run, the response of each endpoint or each test case (the status code, the response format) will be manually analyzed.

E. Evaluation of models' performance

This section evaluates the performance of selected Large Language Models (LLMs) in API testing, utilizing **three primary metrics: parameter coverage, path coverage, and test assertion**. The performance of the models was assessed across diverse prompting methods: **Chain-of-Thought (CoT), Few Shot, One Shot, and Zero Shot**. The compiled APIs encompassed a variety of complexities, including those featuring Create, Retrieve, Update (CRU) endpoints and Get-only APIs, as stated in Table 1, establishing robust benchmarks.

Path coverage

Path coverage measures the ability of LLMs to generate test cases that target API endpoints, including available operations and their sequences.

As seen in Figure 3, **CoT prompting** resulted in the **highest total path coverage** at 24.846, with all models contributing relatively evenly: Grok (6.462), Claude (6.769), Gemini (5.615), and GPT (6.00). The **coverage generally decreased from CoT to Zero Shot prompting** across all 4 models. Zero Shot achieved the lowest total coverage at 20.231. Across all different prompting approach, Claude and Grok showed better performance than Gemini and GPT. CoT proved to be the most effective in covering API paths suggesting that encouraging step-by-step reasoning aids in exploring more API paths.

Parameter coverage

Parameter coverage quantifies the extent to which LLMs generate test cases with appropriate values for required parameters as defined in the API documentation.

As described in Figure 4, **CoT prompting** generally yielded the **highest overall parameter coverage** at 49.846. In this approach, Claude and Grok contributed the most, with 15.769 and 14.462 respectively. **One Shot prompting** achieved the **second-highest** total coverage (47.154). **Few Shots prompting** resulted in a total coverage of 45.769. **Zero Shot prompting** consistently showed the **lowest parameter coverage** (42.077), indicating that providing examples or step-by-step reasoning significantly enhances the LLMs' ability to correctly parameterize test cases.

Test assertion coverage

Test assertion coverage evaluates the LLMs' capability to generate appropriate assertions for verifying API responses, such as status codes or response formats.

Test assertion reveals distinct performance pattern as depicted in Figure 5. Unlike Path and Parameter coverage, test assertion is achieved the **best with Zero Shot prompting in general**. **Claude consistently demonstrated superior performance in generating test assertions** across all prompting approaches. Its contribution far surpassed other LLMs, particularly in **Zero Shot prompting**, where it achieved 39.692 out of a total of 58.538, highlighting its robust capability even without explicit examples while GPT generated only a few assertions. **GPT notably struggled with test assertion generation**, yielding only 0.615 and 0.846 in One Shot and COT prompting. However, even for GPT, it can be seen that **test assertion works the best with no example in Zero Shot Prompting**, achieving its personal best performance with 3.846. **Grok** provided a **consistent, although lower, contribution pattern** to assertions compared to Claude, gradually increasing from 5.857 in Zero Shot to 9.143 with **CoT** prompting, showing a different pattern with other models.

TABLE I. API COLLECTIONS

API Collection	Endpoint Metrics			
	Total Paths (endpoint)	*Parameter*	*Parameter values*	*API Type*
Health Calculator	6	16	25	CRU[a]
Recipe	7	16	16	CRU
Movies DB	7	17	17	CRU
Reqres	7	5	5	CRU
Guardian	5	20	20	Get Only
Local Business Data	6	30	30	Get Only
Twitter	6	9	16	Get Only
NBA Player	8	21	21	Get Only
NoctuaSky	10	47	47	CRU
Fav Q	6	5	5	CRU
NBA Player	8	21	21	Get Only
WynnCraft API	8	6	6	Get Only
Ticket Master	6	8	12	Get Only
TheirStack API	13	56	56	Get Only

[a] CRU refers to Create, Read, Update operations in endpoints

V. RESULTS AND CONSIDERATION

As seen in Figure 6, **all models** under study can achieve similar results in, **nearly 100%, path coverage** which proves that LLM to be efficient in API testing with Open API

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on April 23,2026 at 16:58:20 UTC from IEEE Xplore. Restrictions apply.

document in basic scenarios. Figure 3 and 4 stated that **Claude showed the superior performance** in path and parameter coverage across all prompting approaches. Claude also showed the highest ability in test assertion, outperforming all other models especially with Zero Shot Prompting. Grok follows the second and **Gemini and GPT has relatively lower performance** in all three metrics.

It can be noticed that **COT prompting**, guiding the thinking process of LLMs, significantly improved their code generation ability in path and parameter coverage. However, this approach hinders the models' performance in test assertion possibly due to the emphasized context given to LLMs on understanding the API flow and generation of appr-opriate parameter values. The second reason could be as the context in prompt increases; the token limitation of free models is reached leading to poor performance in response since test assertion needs substantial lines of codes to be generated. Generally, it can be concluded that the prompt with rich and specific context of API under test can improve the performance of models for automated testing, which can be further improved in proprietary versions of LLMs with less token limitations.

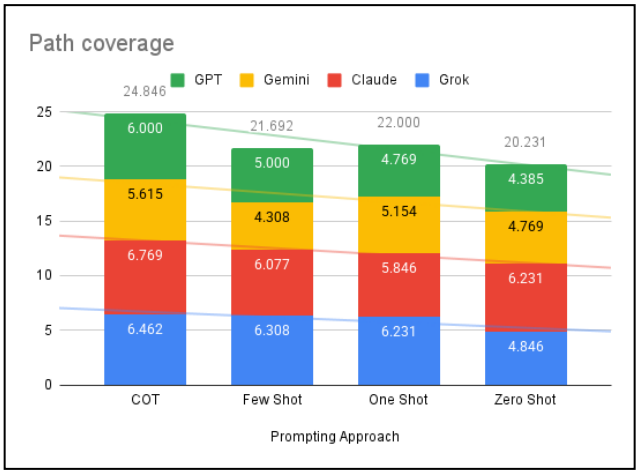


Fig. 3. Path coverage comparison

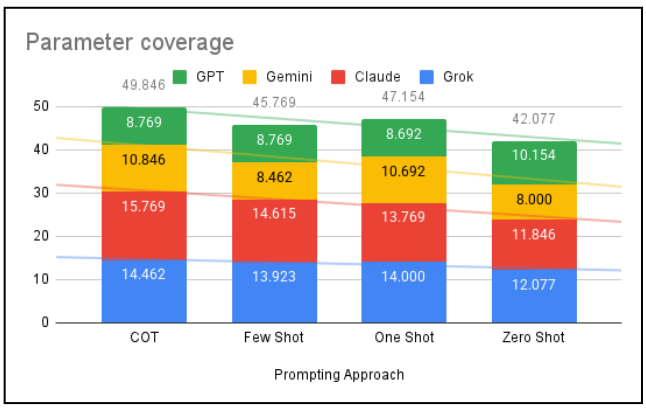


Fig. 4. Parameter coverage comparison

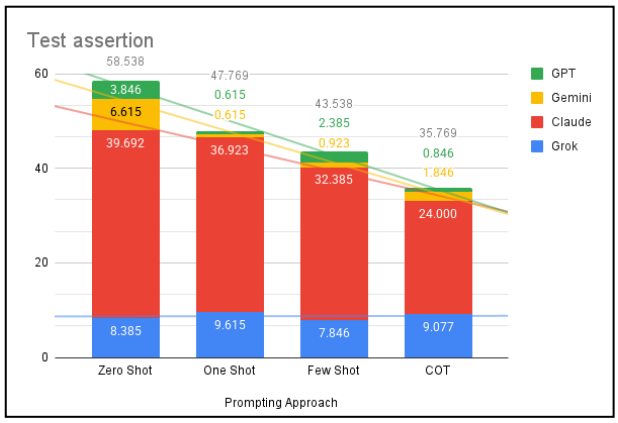


Fig. 5. Test assertion coverage comparison

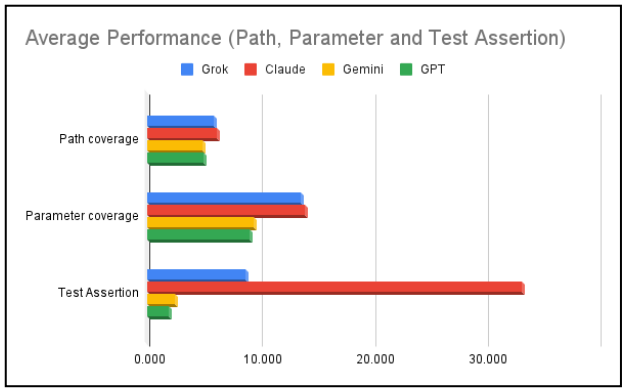


Fig. 6. Average Performance of LLMs across different prompting method

VI. Conclusion

The study comprehensively evaluated the performane of 4 state-of-the-arts LLMs in software API testing, using black-box approach with Open API document of 14 public API services. The results underscore the varying strenghts of different LLMs and the crucial role of prompt engineering in optimising their performance for automated testing. The study highlighted that even without model fine-tuning, public versions of LLMs can perform well with a good quality prompt template in testing of complex API in industry.

Future work entails comparing the performance between LLM generated test cases and existing automated API testing tools, and exploring an optimal prompt template that can be utilized for various APIs in the industry.

Acknowledgment

This study was partially supported by Chulalongkorn University's Graduate Scholarship Program for ASEAN and Non-ASEAN countries.

References

[1] Q. G. M. C. Mustafa Almutawa, "Towards LLM-Assisted System Testing for Microservices," in 2024 IEEE 44th International Conference on Distributed Computing Systems Workshops (ICDCSW), Jersey City, NJ, USA, 2024.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on April 23,2026 at 16:58:20 UTC from IEEE Xplore. Restrictions apply.

[2] P. G. M. P. Vaggelis Atlidakis, "RESTler: Stateful REST API Fuzzing," in 2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE), Montreal, QC, Canada, 2019.

[3] J. L. C. I. J. C. Hamza Ed-douibi, "Automatic Generation of Test Cases for REST APIs: A Specification-Based Approach," in 2018 IEEE 22nd International Enterprise Distributed Object Computing Conference (EDOC), Stockholm, Sweden, 2018.

[4] D. C. S. S. A. O. M. P. R. T.-B. M. C. Myeongsoo Kim, "Enhancing REST API Testing with NLP Techniques," in ISSTA 2023: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis, Seattle WA USA, 2023.

[5] A. U. K. S. Q. M. S. S. A. M. U. N. A. N. B. A. M. Humza Naveed, "A Comprehensive Overview of Large Language Models," 17 October 2024. [Online]. Available: https://arxiv.org/abs/2307.06435v9. [Accessed 29 May 2025].

[6] K. S. F. N. Betim Sherifi, The Potential of LLMs in Automating Software Testing: From Generation to Reporting, 2024.

[7] C. L. J. S. R. P. Vasudev Vikram, Can Large Language Models Write Good Property-Based Tests?, 2024.

[8] M. M. Mohammed Mudassir, "The role of APIs in modern software development," World Journal of Advanced Engineering Technology and Sciences, , vol. 13, no. 01, p. 1045–1047, 2024.

[9] J. G. C. M. M. P. Alexander Lercher ∗, "Microservice API Evolution in Practice: A Study on Strategies and Challenges," The Journal of Systems & Software, vol. 215, no. 112110, 2024.

[10] R. T. Fielding, "Architectural Styles and the Design of Network-based Software Architectures," Ph.D Dissertation, UNIVERSITY OF CALIFORNIA, IRVINE, 2000.

[11] Smart Bear, "Swagger," Smart Bear, [Online]. Available: https://swagger.io/specification/. [Accessed 29 May 2025].

[12] S. S. A. R.-C. Alberto Martin-Lopez, "RESTest: automated black-box testing of RESTful web APIs," in ISSTA 2021: Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Denmark, 2021.

[13] J. G. B. M. M. Z. Andrea Arcuri, "EvoMaster: A Search-Based System Test Generation Tool," The Journal of Open Source Software, vol. 6, no. 2153, 2021.

[14] S. S. A. R.-C. Alberto Martin-Lopez, "Test coverage criteria for RESTful web APIs," in A-TEST 2019: Proceedings of the 10th ACM SIGSOFT International Workshop on Automating TEST Case Design, Selection, and Evaluation, Tallinn Estonia, 2019.

[15] K. S. a. ,. S. K. b. ,. M. B. a. ,. D. D. a. ,. F. F. b. ,. U. G. a. ,. G. G. a. ,. S. G. a. ,. E. H. b. ,. S. K. a. ,. G. K. b. ,. T. M. Enkelejda Kasneci a, "ChatGPT for good? On opportunities and challenges of large language models for education," Science Direct, vol. 103, no. 102274, pp. 1-9, 2023.

[16] M. S. H. M. 2. ,. K. F. 3. N. M. F. S. S. 1. ,. M. M. J. M. 1. J. A. M. E. A. 4. ,. A. S. A. MOHAIMENUL AZAM KHAN RAIAAN 1, "A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges," IEEE Access, vol. 12, no. 3365742, pp. 26839 - 26874, 2024.

[17] M. •. L. H. M. M. •. J. E. M. •. K. D. H. M. •. R. M. M. •. G. S. M. •. L. M. M. t. C. f. D. S. N. Y. U. 6. 5. A. N. Y. N. 1. (. Yiqiu Shen and D. o. Radiology, "ChatGPT and Other Large Language Models AreDouble-edged Swords," Radiology, pp. 1-4, 23 January 2023.

[18] D. M. ,. O. Z.-T. a. A. M. Denis Federiakin *, "Prompt engineering as a new 21st century skill," frontiers, vol. 9, pp. 01 - 05, 2024.

[19] S. P. Hemil Patel, Prompt Engineering For Large Language Model, Research Gate, 2024.

[20] L. Giray, "Prompt Engineering with ChatGPT: A Guide for Academic Writers," Springer Nature Link, vol. 51, no. Annals of Biomedical Engineering, p. 2629–2633, 2023.

[21] A. K. S. ,. S. S. ,. V. J. ,. S. M. a. A. C. Pranab Sahoo1, "A Systematic Survey of Prompt Engineering in Large Language Models: Techniques and Applications," Research Gate, 2024.

[22] A. Z. M. P. M. C. Davide Corradini, "RestTestGen: An Extensible Framework for Automated Black-box Testing of RESTful APIs," in 2022 IEEE International Conference on Software Maintenance and Evolution (ICSME), Limassol, Cyprus, 2022.

[23] *. D. F. R. G. a. D.-I. C. Ovidiu Banias, "Automated Specification-Based Testing of REST APIs," Sensors, vol. 21, no. 16, p. 5375, 2021.

[24] S. S. A. R.-C. Alberto Martin-Lopez, "RESTest: Black-Box Constraint-Based Testing of RESTful Web APIs," in 18th International Conference, ICSOC 2020, Dubai, United Arab Emirates, December 14–17, 2020, Proceedings, Dubai, United Arab Emirates, 2020.

[25] C. L. R. P. Vasudev Vikram, "Can Large Language Models Write Good Property-Based Tests," in ACM Conference, New York, 2023.

[26] P. L. H. W. ,. J. C. ,. W. E. W. Yihao Li, "Evaluating large language models for software testing," Computer Standards & Interfaces, vol. Volume 93, no. 103942, 2024.

[27] I. G. Mike Ralphson, "API Gurus," 2023. [Online]. Available: https://apis.guru/. [Accessed 29 March 2025].

[28] Git Hub, "Git Hub Public APIs," GitHub, Inc, 2025. [Online]. Available: https://github.com/public-apis/public-apis?tab=readme-ov-file#health. [Accessed 29 March 2025].

[29] Rapid API, "Free Public APIs for Developers," Nokia, 2025. [Online]. Available: https://rapidapi.com/collection/list-of-free-apis. [Accessed 29 March 2025].

[30] "Public APIs," 2025. [Online]. Available: https://publicapis.dev/. [Accessed 18 August 2025].

[31] A. Evangelist, "Postman to Open API collection," Postman, [Online]. Available: https://www.postman.com/api-evangelist/artificial-intelligence/documentation/txy0rdd/postman-to-openapi. [Accessed 30 March 2025].

[32] J. A. J. B. Nuno Laranjeiro, "A Black Box Tool for Robustness Testing of REST Services," IEEE Access, vol. 9, pp. 24738 - 24754, 2021. M. Young, The Technical Writer's Handbook. Mill Valley, CA: University Science, 1989.

Authorized licensed use limited to: Universidad Nacional de Ingeniería. Downloaded on April 23,2026 at 16:58:20 UTC from IEEE Xplore. Restrictions apply.